SERVI-X Modelado de la Solución

(Versión 3 — ERS, Propiedad Intelectual, Contrato de Licencia, Trazabilidad, Pruebas de Integración de Caja Negra con evidencia real y Script SQL)

Jose Alvarez Marin

Facultad de Ingeniería, Corporación Universitaria de Sabaneta
Integración de Aplicaciones y Sistemas

Dr. Mauricio Fabian Agudelo
12 de Noviembre de 2026 (actualización de la versión del 17 de octubre de 2026)

Resumen

Este documento presenta el modelado técnico detallado para el módulo SERVI-X, responsable de la gestión integral de Peticiones, Quejas, Reclamos y Sugerencias (PQRS) dentro del ecosistema de microservicios INTEGRAX. A través de un enfoque de diseño riguroso que abarca el Modelo de Datos (Entidad-Relación) y el modelado UML (Casos de Uso, Clases y Secuencia), se define la estructura lógica, el almacenamiento y el comportamiento dinámico del sistema. El diseño propuesto garantiza el aislamiento de datos propio de la arquitectura del proyecto, prioriza la trazabilidad inmutable de las actuaciones (auditoría) y establece los mecanismos de integración asíncrona y síncrona vía API REST con los módulos externos CRM-X, SALES-X y LOGISTI-X.

Nota sobre esta actualización (versión 3): respecto a la versión anterior (17 de octubre de 2026), este documento: (i) unifica la numeración de los requisitos funcionales (sección 1.5) con la de la Matriz de Trazabilidad (sección 9), que antes usaban dos esquemas incompatibles; (ii) ajusta HU-09 y HU-10 para reflejar el comportamiento real implementado —el cierre de caso reutiliza la Respuesta Oficial ya exigida por RN-03 en vez de pedir una descripción de cierre duplicada, y la respuesta oficial toma el correo del cliente automáticamente del caso en vez de pedírselo al agente—; (iii) reemplaza la sección 10 (Pruebas de Integración) por diez pruebas de **caja negra** ejecutadas realmente sobre la aplicación funcional (frontend + API + PostgreSQL), cada una con su captura de pantalla real; y (iv) agrega la sección 11 con el script SQL completo del esquema de base de datos tal como quedó implementado. El modelado UML, el MER y el diagrama de secuencia se conservan sin alteraciones de fondo.

Alcance vigente del módulo: en esta fase, SERVI-X opera de forma **aislada**, con frontend, API REST y base de datos PostgreSQL ya funcionando de extremo a extremo (ver sección 10). Las integraciones con los servicios externos del ecosistema (CRM-X, SALES-X y LOGISTI-X) están diseñadas en el modelo de datos mediante llaves fóraneas lógicas (*cliente_id*, *pedido_id*, *despacho_id*), pero su consumo real vía HTTP/REST queda **planificado para una fase posterior**.

Diagrama de Gantt y Metodología de Trabajo

El proyecto se gestiona con una **metodología híbrida**: el cronograma general de INTEGRAX fija hitos semanales de entrega obligatoria (viernes de cada semana, del 21 de agosto al 18 de septiembre de 2026 según el cronograma oficial del curso), mientras que el trabajo dentro de cada semana se organiza de forma ágil mediante ramas de feature en Git (*feature/** -> *dev* -> *main*), con aprobación explícita antes de cada integración. Esto permite entregar incrementos funcionales verificables en cada hito sin perder la trazabilidad de los cambios.

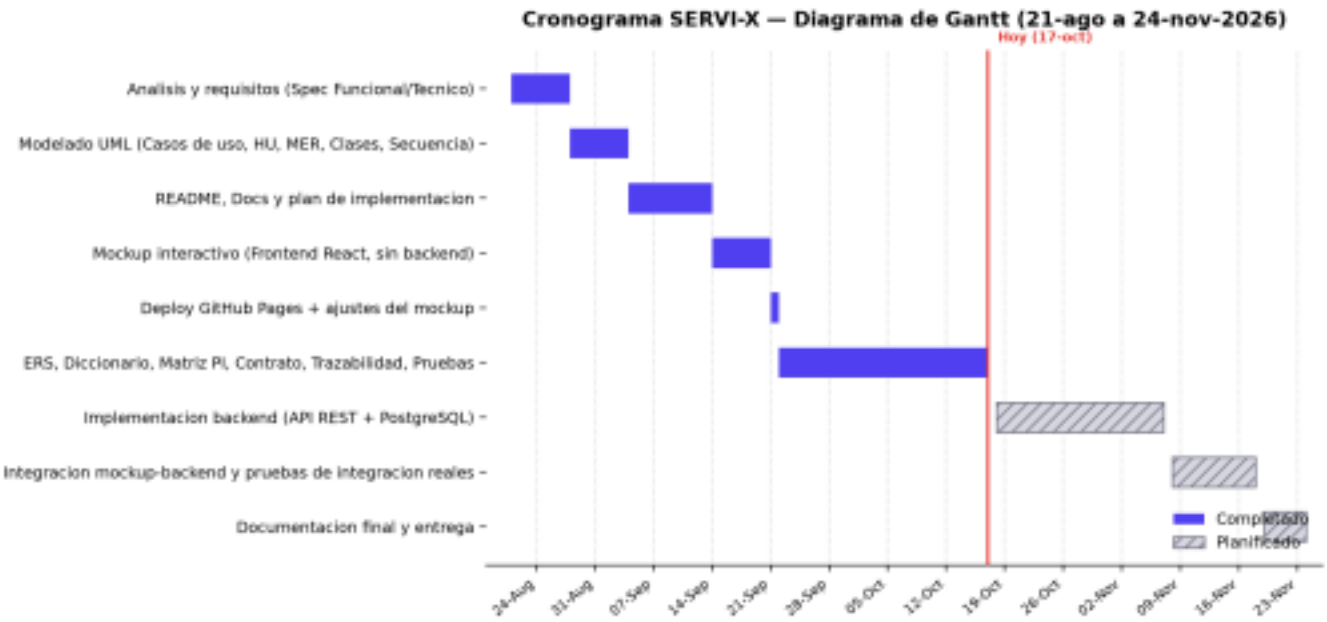


Figura 0. Diagrama de Gantt del proyecto SERVI-X — fases completadas y planificadas.

Las primeras seis fases (análisis, modelado UML, documentación de base y mockup interactivo desplegado en GitHub Pages) ya se completaron. La implementación del backend con PostgreSQL ya se completó y se verificó de extremo a extremo (ver sección 10): la aplicación completa —frontend, API REST y base de datos— está funcional.

1. Especificación de Requisitos de Software (ERS — IEEE 830)

1.1 Introducción

La presente Especificación de Requisitos de Software (ERS) documenta de manera formal las necesidades funcionales y no funcionales del módulo SERVI-X, siguiendo la estructura recomendada por el estándar IEEE 830. Su propósito es servir como contrato técnico entre el equipo desarrollador y los interesados del proyecto académico INTEGRAX, garantizando una base verificable para el diseño, la implementación y las pruebas del sistema.

1.2 Propósito

El propósito de este documento es especificar, sin ambigüedad, el comportamiento esperado del módulo SERVI-X: qué debe hacer el sistema (requisitos funcionales), bajo qué condiciones de calidad debe operar (requisitos no funcionales), y qué restricciones técnicas, reglas de negocio y criterios de aceptación enmarcan su desarrollo.

1.3 Alcance

SERVI-X es el módulo encargado de centralizar, clasificar, asignar, atender, responder y cerrar las solicitudes de PQRS de los clientes de la plataforma INTEGRAX, garantizando trazabilidad inmutable de cada actuación. El sistema ya opera de forma funcional completa: frontend en React, API REST en Node.js/TypeScript y base de datos PostgreSQL propia, verificado mediante las pruebas de la sección 10.

Aclaración de alcance: las integraciones salientes hacia CRM-X, SALES-X y LOGISTI-X están fuera del alcance funcional de esta entrega: se reserva el modelo de datos para ellas (campos cliente_id, pedido_id, despacho_id como llaves foráneas lógicas), pero su consumo real vía HTTP/REST queda planificado para una integración posterior dentro del cronograma general de INTEGRAX.

1.4 Descripción general

1.4.1 Perspectiva del producto: SERVI-X es un sistema web compuesto por un frontend en React (Vite) y una API REST en Node.js/TypeScript (Express), respaldada por una base de datos relacional PostgreSQL. Es uno de los seis microproyectos independientes que conforman INTEGRAX y está diseñado bajo una Arquitectura Orientada a Servicios (SOA) para integrarse posteriormente con los módulos restantes.

1.4.2 Funciones principales: registro y autenticación de usuarios; radicación, clasificación, consulta y seguimiento de PQRS; asignación de responsables; actualización de estados con trazabilidad obligatoria; registro de respuestas oficiales; cierre controlado de casos; generación de reportes de gestión.

1.4.3 Usuarios del sistema:

- **Cliente:** radica solicitudes, consulta sus propias solicitudes y su historial, y visualiza las respuestas oficiales.
- **Agente de servicio:** clasifica, asigna, gestiona el estado, responde, cierra casos y genera reportes de gestión.
- **Administrador:** además de las funciones de Agente, gestiona (crea, edita, desactiva/reactiva) los perfiles operativos del módulo.

1.5 Requisitos Funcionales

Los requerimientos funcionales se numeran de forma **alineada uno a uno con los 10 casos de uso** (sección 3) y con la Matriz de Trazabilidad (sección 9), para evitar la inconsistencia de numeración que existía en la versión anterior de este documento. Los requisitos RF-11 y RF-12 son transversales de integración empresarial y no tienen un caso de uso dedicado.

ID	Nombre del Requerimiento	Descripción Detallada	UC
RF-01	Crear usuario	Permite el autorregistro de Clientes, generando un identificador único (UUID v4) e iniciando su ciclo de vida en el sistema.	UC-01
RF-02	Ingresar	Autentica a Clientes y Agentes, emitiendo un JSON Web Token con el rol del usuario para controlar el acceso.	UC-02
RF-03	Consultar solicitudes	Habilita la visualización de un listado de casos con sus estados actuales, con búsqueda por número de solicitud y filtros por fecha, estado y tipo; para un Cliente, filtra automáticamente por su propio identificador.	UC-03
RF-04	Consultar historial de atención	Muestra la línea de tiempo inmutable de un caso, detallando fechas, responsables y transiciones de estado extraídas de la tabla de auditoría.	UC-04
RF-05	Registrar PQRS	Permite crear una nueva solicitud clasificándola según su tipología exacta (Petición, Queja, Reclamo, Solicitud, Felicitación u Observación), con pedido o despacho asociados de forma opcional.	UC-05
RF-06	Asignar responsables	Permite vincular un agente operativo activo (responsable_id) a un caso abierto para gestionar su resolución.	UC-06
RF-07	Actualizar estado de atención	Permite modificar la etapa del caso, exigiendo justificación y registrando la transición (estado anterior/nuevo) en la bitácora.	UC-07

ID	Nombre del Requerimiento	Descripción Detallada	UC
RF-08	Generar reportes de gestión	Consolida métricas operativas (cantidad de casos por tipo de solicitud y por estado de atención) consultando directamente la base de datos aislada.	UC-08
RF-09	Cerrar casos	Finaliza el ciclo de vida de la PQRS únicamente si existe una respuesta oficial previa (RN-03), estampando la fecha de cierre y bloqueando el registro contra modificaciones futuras.	UC-09
RF-10	Registrar respuestas	Habilita la redacción e inserción de la respuesta formal que se brindará al cliente como solución a su solicitud.	UC-10
RF-11	Consultar información asociada a pedidos y despachos	Permite a SERVI-X consumir APIs de módulos externos (SALES-X, LOGISTI-X) para enriquecer el detalle de un caso. (Integración planificada para fase posterior; no activa en esta entrega.)	—
RF-12	Exponer servicios para integración empresarial	Provee endpoints para que otros módulos del ecosistema INTEGRAX u otras plataformas externas puedan radicar o consultar casos.	—

1.6 Requisitos No Funcionales

ID	Categoría	Descripción Detallada
RNF-01	Disponibilidad	El módulo SERVI-X debe garantizar un tiempo de operatividad mínimo del 95%.
RNF-02	Rendimiento	Toda petición estándar realizada al backend debe procesarse en un tiempo inferior a 3 segundos (3000 ms).
RNF-03	Usabilidad	La interfaz de usuario debe ser intuitiva, adaptándose a distintos tamaños de pantalla.
RNF-04	Seguridad	El sistema debe validar la identidad mediante JWT y restringir el acceso según los roles (Cliente, Agente, Administrador).
RNF-05	Auditabilidad	Persistencia de una bitácora inmutable (tabla historial_atencion) que impida la sobrescritura del pasado.
RNF-06	Arquitectura	Toda la comunicación entre el frontend, SERVI-X y los demás módulos debe realizarse bajo los principios REST.
RNF-07	Portabilidad	Las vistas deben renderizarse correctamente en navegadores de escritorio y dispositivos móviles.

ID	Categoría	Descripción Detallada
RNF-08	Escalabilidad	La base de datos y los servicios deben soportar el incremento transaccional, apoyándose en UUID v4.

1.7 Restricciones

- Lenguaje y entorno de ejecución: Node.js 20+ con TypeScript.
- Framework web: Express.js.
- Frontend: React con Vite.
- Motor de base de datos: PostgreSQL, exclusivo del módulo (sin tablas compartidas con otros microproyectos).
- Todas las llaves primarias y foráneas deben ser UUID v4; queda prohibido el uso de enteros autoincrementales.
- Nomenclatura de base de datos en snake_case; contratos de API en camelCase.

1.8 Interfaces

- **Interfaz de usuario:** aplicación web responsiva (portal Cliente, portal Agente y portal Administrador), accesible desde navegador en escritorio y móvil.
- **Interfaz de hardware:** ninguna especial; el sistema corre sobre infraestructura de servidor virtualizada/en contenedores.
- **Interfaz de software:** API REST en formato JSON como único canal de comunicación entre el frontend y el backend.

1.9 Reglas de Negocio

ID	Regla
RN-01	Todo identificador (caso, usuario, historial) debe ser UUID v4, nunca secuencial.
RN-02	SERVI-X opera sobre su propio repositorio de datos; debe seguir funcionando aunque otros módulos del ecosistema fallen.
RN-03	No se puede cerrar un caso si no existe al menos una respuesta oficial registrada previamente.

ID	Regla
RN-04	Todo caso nuevo debe generar simultáneamente su primer registro en la bitácora de auditoría.

1.10 Criterios de Aceptación

- El sistema permite registrar, autenticar, radicar, asignar, actualizar, responder, cerrar y consultar PQRS cumpliendo los 10 casos de uso definidos —verificado en la sección 10 con pruebas de caja negra reales.
- Ninguna operación de cierre se ejecuta sin respuesta oficial previa (RN-03), verificado con prueba de integración CP-06.
- La tabla historial_atencion rechaza operaciones UPDATE/DELETE a nivel de base de datos (trigger, ver sección 11).
- Los contratos JSON usan camelCase y las columnas de base de datos usan snake_case de forma consistente.

2. Modelo de Entidad-Relación

El modelo de datos relacional define la estructura de almacenamiento exclusiva y aislada para el módulo SERVI-X. Para asegurar la escalabilidad dentro del ecosistema de microservicios, el esquema utiliza identificadores universales (UUID v4) como llaves primarias, previniendo colisiones de datos en la red, y aplica estrictamente el estándar snake_case. El modelo garantiza la integridad de la información mediante las siguientes entidades principales y sus relaciones de cardinalidad:

Cliente: usuario final que radica solicitudes; se autorregistra (UC-01).

Caso (Entidad Central): Es el núcleo transaccional de la PQRS. Almacena los atributos descriptivos (tipo de solicitud, estado) y las referencias externas (cliente, pedido, despacho).

Historial_Atencion (Trazabilidad y Auditoría): Bitácora inmutable diseñada para registrar cualquier transición o actuación sobre una solicitud, almacenando el estado anterior, el nuevo y la respuesta dada.

- **Cardinalidad Cliente (1) a Caso (N):** un cliente puede radicar múltiples casos a lo largo del tiempo.
- **Cardinalidad Caso (1) a Historial (N):** un solo caso puede acumular múltiples actuaciones a lo largo del tiempo, pero cada registro histórico pertenece única y exclusivamente a un caso específico.

Responsable (Gestión de Actores): Entidad paramétrica que administra los datos de los agentes internos (nombre, rol, email, estado).

- **Cardinalidad Responsable (1) a Caso (N):** un agente puede tener asignados múltiples casos simultáneamente, mientras que un caso, en un momento dado, sólo tiene un único responsable a cargo.
- **Cardinalidad Responsable (1) a Historial (N):** un agente puede generar múltiples registros de actualización en el tiempo, permitiendo identificar de forma inequívoca qué empleado ejecutó cada acción.

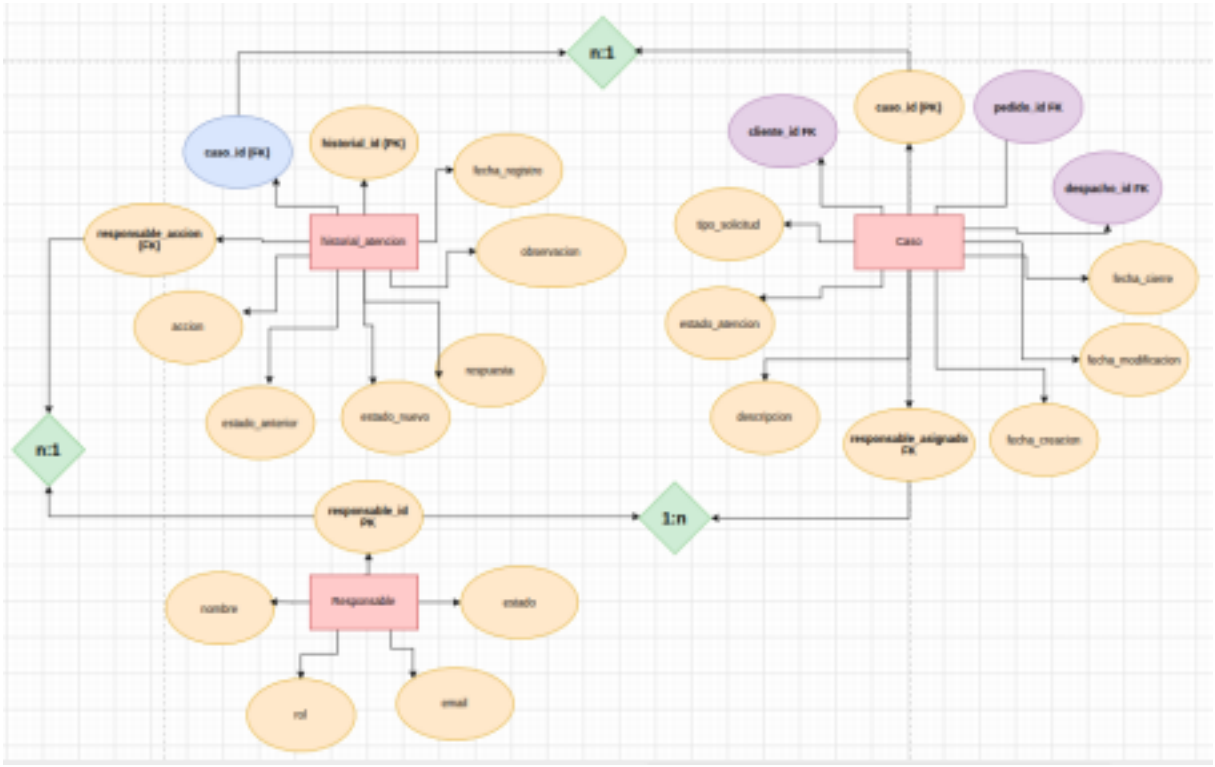


Figura 1. Modelo Entidad-Relación de SERVI-X.

3. Modelo de Casos de Uso

A continuación, se define el comportamiento lógico, las condiciones y el flujo de los casos de uso representados en el modelo UML para el módulo SERVI-X:

ID	Nombre del Caso	Actor	Descripción y Flujo Principal	Precondición
UC-01	Crear usuario	Cliente	Registro de nuevos clientes: nombre, tipo de documento, número de documento, email y contraseña.	Ninguna.
UC-02	Ingresar	Cliente, Agente	Autenticación mediante usuario y contraseña para obtener acceso según el rol.	Haber creado un usuario previamente.
UC-03	Consultar solicitudes	Cliente, Agente	Visualización del listado de PQRS, con búsqueda por número y filtros por fecha, estado o tipo.	El usuario debe estar autenticado.
UC-04	Consultar historial	Cliente, Agente	Línea de tiempo de un caso, con filtros por fecha, estado y tipo, y vista de detalle.	Conocer el ID del caso y estar autenticado.
UC-05	Registrar PQRS	Cliente, Agente	Creación de una nueva solicitud: Petición, Queja, Reclamo, Solicitud, Felicitación u Observación, con pedido/despacho asociados opcionalmente.	Estar autenticado.
UC-06	Asignar responsables	Agente	Selección de un caso y asignación de un agente operativo.	El caso debe estar en estado “Radicado” o “Abierto”.
UC-07	Actualizar estado	Agente	Modificación de la etapa del caso en progreso, con trazabilidad.	El caso debe estar asignado a un responsable.
UC-08	Generar reportes	Agente	Consolidación de métricas filtrando por fecha, estado y tipo de solicitud.	Tener historial registrado en el sistema.
UC-09	Cerrar casos	Agente	Finalización del ciclo de vida de la PQRS, únicamente cuando ya existe una respuesta oficial registrada (RN-03).	El caso debe tener una respuesta registrada.
UC-10	Registrar respuestas	Agente	Redacción de la respuesta formal al cliente, asociada directamente al caso.	El caso debe estar en proceso de investigación.



Figura 2. Modelo de Casos de Uso de SERVI-X.

3.1 Historias de Usuario

HU-01: Crear usuario

Como Cliente
Quiero registrarme en el sistema SERVI-X
Para poder radicar PQRS y consultar el estado de mis solicitudes.

Criterios de aceptación:

- El formulario en React debe solicitar: nombre completo, tipo de documento, número de documento, correo electrónico y contraseña.
- El sistema debe validar que el correo electrónico no se encuentre previamente registrado.
- La contraseña debe almacenarse encriptada mediante bcrypt antes de persistirse.
- El backend debe generar automáticamente un identificador único UUID v4 para el nuevo cliente.
- El registro debe persistirse en la base de datos relacional PostgreSQL aislada del módulo.
- La petición HTTP debe procesarse y retornar un código de éxito en formato JSON en menos de 3000 ms (RNF-02).

HU-02: Ingresar

Como Cliente o Agente de servicio
Quiero autenticarme en el sistema SERVI-X
Para acceder a las funcionalidades correspondientes según mi rol operativo.

Criterios de aceptación:

- El sistema debe validar las credenciales contra la base de datos PostgreSQL aislada del módulo.
- El backend debe generar un JSON Web Token (JWT) al autenticar exitosamente al usuario.
- El payload del JWT debe incluir el rol del usuario para el control de acceso (RNF-04).
- Si las credenciales son incorrectas, la API debe retornar HTTP 401 sin revelar detalles específicos.
- El frontend debe almacenar el token y redirigir al usuario a su panel correspondiente.

HU-03: Consultar solicitudes

Como Cliente o Agente de servicio
Quiero consultar el listado de PQRS registradas
Para visualizar el estado actual de las solicitudes y su información general.

Criterios de aceptación:

- El sistema debe retornar el listado en formato JSON usando camelCase para los contratos de red.
- Si el rol autenticado es Cliente, el backend debe filtrar la consulta para retornar únicamente los registros asociados a su UUID.
- La interfaz debe permitir ingresar el número de solicitud o buscar por nombre del cliente o por descripción del caso para búsqueda directa.
- El listado debe proporcionar filtros funcionales por fecha, estado de atención y tipo de solicitud.
- La vista debe ser responsiva (RNF-07).

HU-04: Consultar historial

Como Cliente o Agente de servicio
Quiero visualizar la bitácora completa de un caso específico
Para auditar la línea de tiempo y todas las transiciones de estado ocurridas.

Criterios de aceptación:

- El endpoint REST debe exigir un identificador de caso válido (UUID v4) en la ruta.
- El backend debe ejecutar una consulta de solo lectura dirigida exclusivamente a la tabla Historial_Atencion.
- El JSON de respuesta debe incluir literalmente los campos: estadoAnterior, estadoNuevo, accionRealizada, usuarioAccionId y fechaRegistro.
- El frontend debe renderizar los datos como una línea de tiempo ordenada cronológicamente.
- El middleware de autenticación debe garantizar que un cliente solo pueda leer historiales asociados a sus propios casos.
- La interfaz debe incluir una opción de “Ver detalles” que expanda la información completa de cada evento.

HU-05: Registrar PQRS

Como Cliente o Agente de servicio
Quiero registrar una nueva PQRS en el sistema
Para escalar un requerimiento operativo, queja, reclamo, solicitud, felicitación u observación formal.

Criterios de aceptación:

- El sistema debe generar automáticamente un identificador único UUID v4 para el caso.
- La interfaz debe permitir seleccionar la tipología exacta: Petición, Queja, Reclamo, Solicitud, Felicitación u Observación.
- El sistema debe permitir opcionalmente registrar un código de pedido o de despacho asociado (texto libre, pendientes de validar vía SALES-X/LOGISTI-X cuando esa integración se habilite).
- El registro inicial debe almacenarse en la base de datos PostgreSQL aislada del módulo.
- Se debe insertar simultáneamente un registro inicial en Historial_Atencion para garantizar la trazabilidad de la radicación (RN-04).

HU-06: Asignar responsables

Como Agente de servicio
Quiero asignar un empleado responsable a un caso abierto
Para garantizar que la solicitud tenga un encargado para su resolución operativa.

Criterios de aceptación:

- El sistema debe consultar la entidad Responsable para listar los agentes activos.
- La interfaz debe permitir elegir el responsable entre los agentes disponibles.
- El backend debe actualizar el atributo de asignación en la tabla Caso mediante la llave UUID del agente.
- Como el caso solo tiene un campo responsable_id (no una lista), el diseño mismo impide tener más de un responsable simultáneo: reasignar sobrescribe al anterior.
- El sistema debe registrar el evento de asignación en la bitácora inmutable de auditoría.

- El acceso a esta funcionalidad debe estar restringido al rol de Agente o Administrador (RNF-04).

HU-07: Actualizar estado

Como Agente de servicio
Quiero actualizar el estado de un caso en progreso
Para reflejar el avance real de la gestión operativa.

Criterios de aceptación:

- El formulario debe exigir que el agente ingrese una justificación de texto al modificar el estado.
- El controlador debe capturar el valor del estado anterior y el estado nuevo.
- El sistema debe actualizar el atributo estadoAtencion en la entidad Caso dentro de PostgreSQL.
- Se debe insertar una fila transaccional en Historial_Atencion que documente ambos estados (RNF-05).
- La actualización exitosa debe notificar a la interfaz cliente sin requerir recargar la página completa.

HU-08: Generar reportes

Como Agente de servicio
Quiero generar reportes de gestión del módulo SERVI-X
Para consolidar métricas operativas que apoyen la toma de decisiones.

Criterios de aceptación:

- El backend debe exponer un endpoint REST específico para la consolidación de métricas de negocio.
- El sistema debe ejecutar consultas de agrupación nativas en PostgreSQL para calcular el volumen total.
- La interfaz debe permitir generar reportes filtrando por rango de fechas.
- Las métricas retornadas deben agrupar la cantidad de casos por estadoAtencion y por tipoSolicitud.

- El payload JSON debe estar formateado para ser renderizado directamente en la interfaz web sin procesamiento adicional.

HU-09: Cerrar casos

Como Agente de servicio
Quiero cerrar un caso que ya tiene respuesta registrada
Para finalizar el ciclo de vida de la PQRS y bloquear modificaciones futuras.

Criterios de aceptación:

- El backend debe rechazar la solicitud de cierre si el caso no tiene al menos una respuesta oficial previa asociada (RN-03).
- El flujo se realiza desde la vista de detalle del caso ya abierto: el agente no debe reingresar el número de caso (ya identificado por la pantalla en la que está) ni una descripción de cierre adicional —la justificación de la resolución es, precisamente, la Respuesta Oficial ya exigida por RN-03; pedir una segunda descripción duplicaría ese mismo dato.
- El sistema debe estampar la fecha y hora exacta del cierre en la entidad principal del caso (fechaCierre).
- El sistema debe bloquear cualquier operación de actualización posterior sobre ese registro de caso.
- Se debe generar una traza final en Historial_Atencion documentando el cierre.
- El estado actualizado debe reflejarse inmediatamente en el módulo de generación de reportes.

HU-10: Registrar respuestas

Como Agente de servicio
Quiero redactar una respuesta formal dentro del caso
Para brindar una solución oficial al requerimiento del cliente.

Criterios de aceptación:

- El sistema debe habilitar un área de texto extenso en el panel operativo para redactar la respuesta.

- El flujo se realiza desde la vista de detalle del caso: el agente solo ingresa la descripción de la respuesta; el correo electrónico del cliente no se vuelve a pedir porque el caso ya tiene asociado su cliente_id, y de ahí el sistema resuelve el correo automáticamente —evita error humano de tipeo que enviaría la respuesta a un correo incorrecto.
- La respuesta ingresada debe quedar asociada al caso correspondiente.
- El texto oficial debe persistirse como un evento inmutable dentro del historial (RNF-05).
- La respuesta debe quedar disponible en modo lectura para el cliente en su panel de consulta.
- El sistema debe validar que exista al menos una respuesta oficial registrada antes de habilitar el cierre del caso.

4. Modelo de Clases y Estructura de Objetos

El diagrama de clases ilustra la vista estática del backend para el módulo SERVI-X. Se adoptó camelCase para todos los atributos y métodos (facilita la serialización JSON) y encapsulamiento estricto (atributos privados, mutación solo mediante métodos públicos).

- **Clase Caso:** objeto central de la lógica de PQRS. Gestiona tipoSolicitud y estadoAtencion; métodos actualizarEstado(nuevoEstado) y cerrarCaso().
- **Clase HistorialAtencion:** objeto transaccional de auditoría; método registrarActualizacion() garantiza traza inmutable.
- **Clase Responsable:** modela al agente operativo; métodos administrativos como actualizarResponsable().
- **Clase Cliente:** modela al usuario final que radica PQRS; se crea vía registrarCliente().

Asociaciones y multiplicidad: Caso 1—* HistorialAtencion; Responsable 1—* Caso; Responsable 1—* HistorialAtencion (como autor de los eventos); Cliente 1—* Caso.

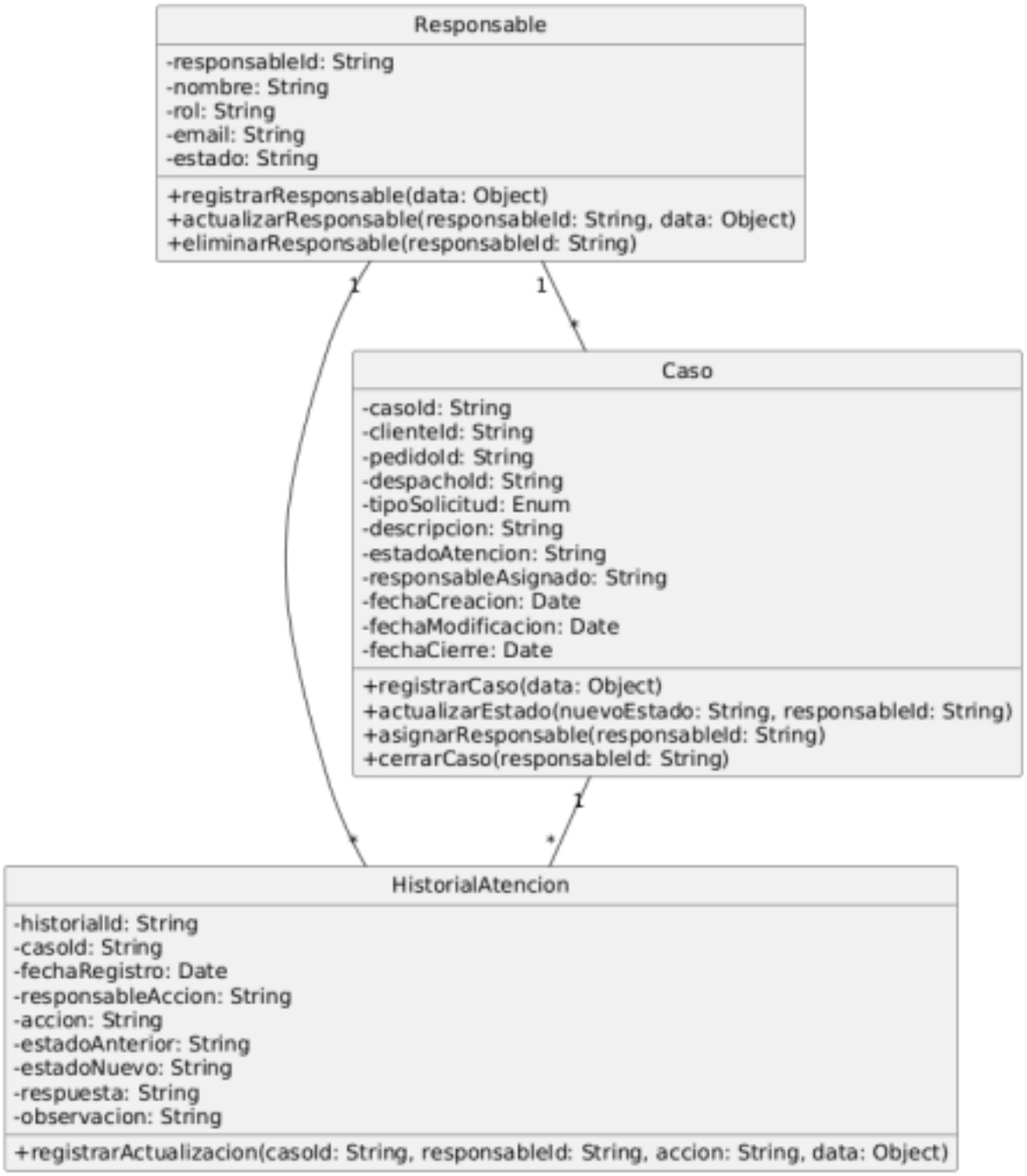


Figura 3. Diagrama de Clases de SERVI-X.

4.1 Diccionario de Datos

Entidad caso

Campo	Tipo	Long.	PK	FK	Oblig.	Descripción y Regla de Negocio
caso_id	UUID	36	Sí	No	Sí	Identificador universal único generado (UUID v4) al radicar.
cliente_id	UUID	36	No	Sí (cliente)	Sí	Llave foránea hacia la tabla cliente; se resuelve automáticamente desde la sesión del Cliente autenticado.
pedido_id	VARCHAR	100	No	Lógica (SALES-X)	No	Código de pedido asociado, texto libre; integración con SALES-X planificada.
despacho_id	VARCHAR	100	No	Lógica (LOGISTI-X)	No	Código de despacho asociado, texto libre; integración con LOGISTI-X planificada.
tipo_solicitud	VARCHAR	50	No	No	Sí	Enumeración: 'Peticion', 'Queja', 'Reclamo', 'Solicitud', 'Felicitacion', 'Observacion'.
estado_atencion	VARCHAR	50	No	No	Sí	Estado actual (ej. 'Radicado', 'En Proceso', 'Cerrado').
descripcion	TEXT	Variable	No	No	Sí	Detalle textual de la necesidad expresada por el cliente.
fecha_creacion	TIMESTAMP	N/A	No	No	Sí	Fecha y hora exacta de la radicación (NOW()).
fecha_cierre	TIMESTAMP	N/A	No	No	No	Se estampa únicamente cuando el caso finaliza su ciclo de vida.
responsable_id	UUID	36	No	Sí (responsable)	No	ID del Agente asignado.

Entidad cliente

Campo	Tipo	Long.	PK	FK	Oblig.	Descripción y Regla de Negocio
cliente_id	UUID	36	Sí	No	Sí	Identificador del cliente (UUID v4).
nombre_completo	VARCHAR	150	No	No	Sí	Nombre y apellidos del cliente.

Campo	Tipo	Long.	PK	FK	Oblig.	Descripción y Regla de Negocio
tipo_documento	VARCHAR	50	No	No	Sí	Tipo de documento de identidad.
numero_documento	VARCHAR	50	No	No	Sí	Número de documento de identidad.
email	VARCHAR	100	No	No	Sí (único)	Correo electrónico y credencial de acceso.
password_hash	VARCHAR	255	No	No	Sí	Contraseña encriptada con bcrypt (salt rounds 10).
fecha_creacion	TIMESTAMP	N/A	No	No	Sí	Fecha de autorregistro.

Entidad historial_atencion

Tabla estrictamente inmutable (solo soporta sentencias INSERT; un trigger de PostgreSQL rechaza UPDATE y DELETE —ver sección 11).

Campo	Tipo	Long.	PK	FK	Oblig.	Descripción y Regla de Negocio
historial_id	UUID	36	Sí	No	Sí	Identificador único del evento registrado (UUID v4).
caso_id	UUID	36	No	Sí (caso)	Sí	Relación obligatoria hacia la tabla caso.
estado_anterior	VARCHAR	50	No	No	No	Estado previo del caso (NULL únicamente en el primer registro).
estado_nuevo	VARCHAR	50	No	No	Sí	Nuevo estado asignado tras la acción del usuario.
accion_realizada	TEXT	Variable	No	No	Sí	Justificación obligatoria o respuesta oficial brindada.
usuario_accion_id	UUID	36	No	Sí (resp./cliente)	Sí	ID del usuario (Cliente o Agente) que ejecutó el cambio.
accion	VARCHAR	50	No	No	Sí	Tipo de evento: Creacion, Asignacion, CambioEstado, RespuestaOficial, Cierre.

Campo	Tipo	Long.	PK	FK	Oblig.	Descripción y Regla de Negocio
fecha_registro	TIMESTAMP	N/A	No	No	Sí	Estampa de tiempo exacta de la transacción.

Entidad responsable

Campo	Tipo	Long.	PK	FK	Oblig.	Descripción y Regla de Negocio
responsable_id	UUID	36	Sí	No	Sí	Identificador del agente (UUID v4).
nombre_completo	VARCHAR	150	No	No	Sí	Nombre y apellidos del funcionario.
correo_corporativo	VARCHAR	100	No	No	Sí (único)	Medio de contacto y credencial de acceso.
password_hash	VARCHAR	255	No	No	Sí	Contraseña encriptada con bcrypt (salt rounds 10).
rol_operativo	ENUM	—	No	No	Sí	'Agente' o 'Administrador'.
estado_activo	BOOLEAN	1	No	No	Sí	Indica si el agente está habilitado (TRUE) para recibir asignaciones.

5. Diagrama de Secuencia

El modelo de secuencia expone el comportamiento dinámico del sistema y el flujo temporal de los mensajes intercambiados entre los distintos componentes del módulo SERVI-X. El diagrama ilustra la interacción interna entre el Frontend, la API REST, la capa de Servicio de Negocio y la Base de Datos PostgreSQL aislada, sin aún conectarse con los servicios externos del ecosistema INTEGRAX.

- **Autenticación y Control de Acceso:** registro con validación de unicidad de correo y hash de contraseña; ingreso con validación de credenciales y emisión de JWT con el rol embebido (RNF-04).
- **Transaccionalidad y Registro de Auditoría:** cada acción relevante ejecuta la inserción simultánea en caso e historial_atencion, garantizando trazabilidad e inmutabilidad (RNF-05).
- **Gestión del Ciclo de Vida de la PQRS:** asignación, clasificación, actualización de estado con justificación, respuesta oficial y cierre con validación condicional de RN-03.
- **Consultas y Reportería:** listados con filtros dinámicos, consulta de la línea de tiempo, y reportes agregados (GROUP BY).
- **Estándares de Interoperabilidad:** respuestas en JSON camelCase; persistencia en PostgreSQL snake_case.

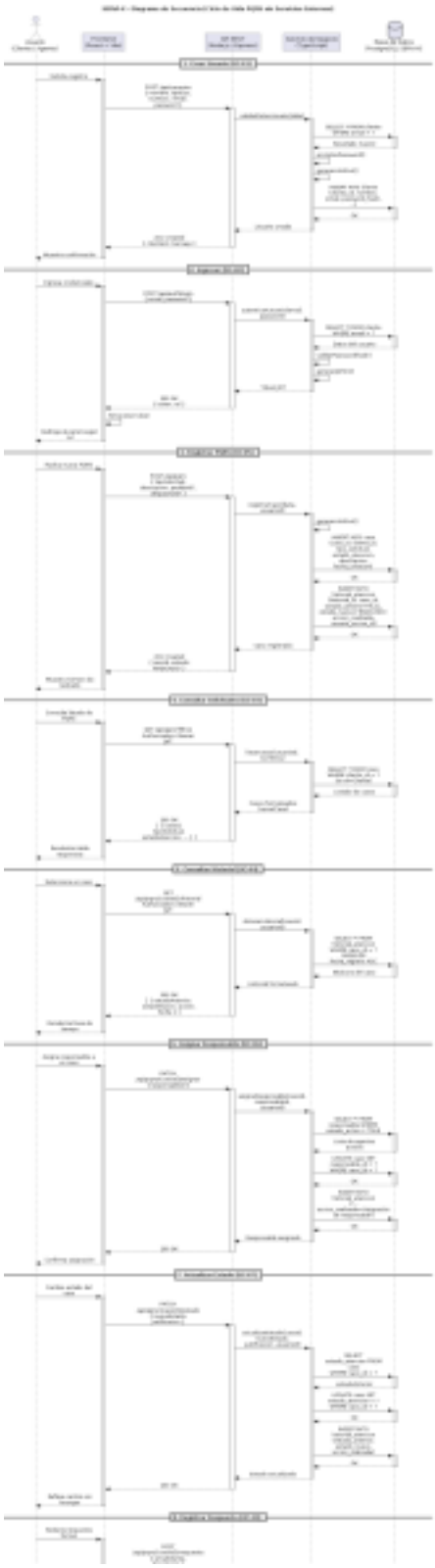


Figura 4. Diagrama de Secuencia — Ciclo de vida de una PQRS (sin servicios externos).

6. Arquitectura

6.1 Ficha técnica de SOFTWARE

Campo	Descripción
Nombre del sistema / proyecto	Módulo SERVI-X (Ecosistema INTEGRAX)
Lenguaje(s) de programación	TypeScript y entorno de Node. Base de datos relacional PostgreSQL
Framework(s) / librerías principales	Express.js para la API REST, Prisma ORM, y React con Vite para el frontend
Motor de base de datos	PostgreSQL 16
Sistema operativo compatible	Multiplataforma 100% web, accesible desde cualquier navegador moderno.
Herramientas de desarrollo	Visual Studio Code, Git/GitHub, Docker, Jest/Supertest.
Tipo de licenciamiento	Propietario / Académico (ver secciones 7 y 8).
Requerimientos funcionales principales	Gestión del ciclo de vida de PQRS, registro histórico inmutable y generación de reportes.
Módulos / servicios de integración	Consumo planificado (no activo en esta fase) de CRM-X, SALES-X y LOGISTI-X.

6.2 Ficha técnica de HARDWARE

Campo	Descripción
Tipo de servidor	Servidor en la nube virtualizado mediante contenedores Docker (Railway planificado).
Procesador	Mínimo 2 vCPU.
Memoria RAM	Mínima 2GB / Recomendada 4GB.
Almacenamiento	Mínimo 20GB SSD.
Requerimientos de red	Ancho de banda que garantice respuestas en menos de 3 segundos.
Dispositivos cliente compatibles	Multiplataforma 100% web.
Proveedor de infraestructura	Despliegue planificado en la nube (Railway), con integración continua desde GitHub.
Mecanismos de respaldo	Backups automáticos planificados vía GitHub Actions.

6.3 Ficha técnica de ARQUITECTURA

Campo	Descripción
Tipo de arquitectura	Arquitectura Orientada a Servicios (SOA) con persistencia de datos relacional y aislada.
Patrón de integración	Comunicación síncrona mediante API REST (consumo externo planificado, no activo en esta fase).
Componentes / capas	Capa de Presentación (React), Capa de Lógica/API (Express + Prisma), Capa de Datos (PostgreSQL aislada).
Protocolos de comunicación	HTTP en formato JSON, camelCase.
Mecanismo de seguridad	JWT con roles (Cliente, Agente, Administrador).
Estrategia de escalabilidad	UUID v4 en todas las llaves.
Servicios o APIs externas	Planificadas: /api/clientes (CRM-X), /api/pedidos (SALES-X), /api/despachos (LOGISTI-X). No activas en esta entrega.
Estrategia de despliegue	Dockerfile para el backend; docker-compose para backend + PostgreSQL local.

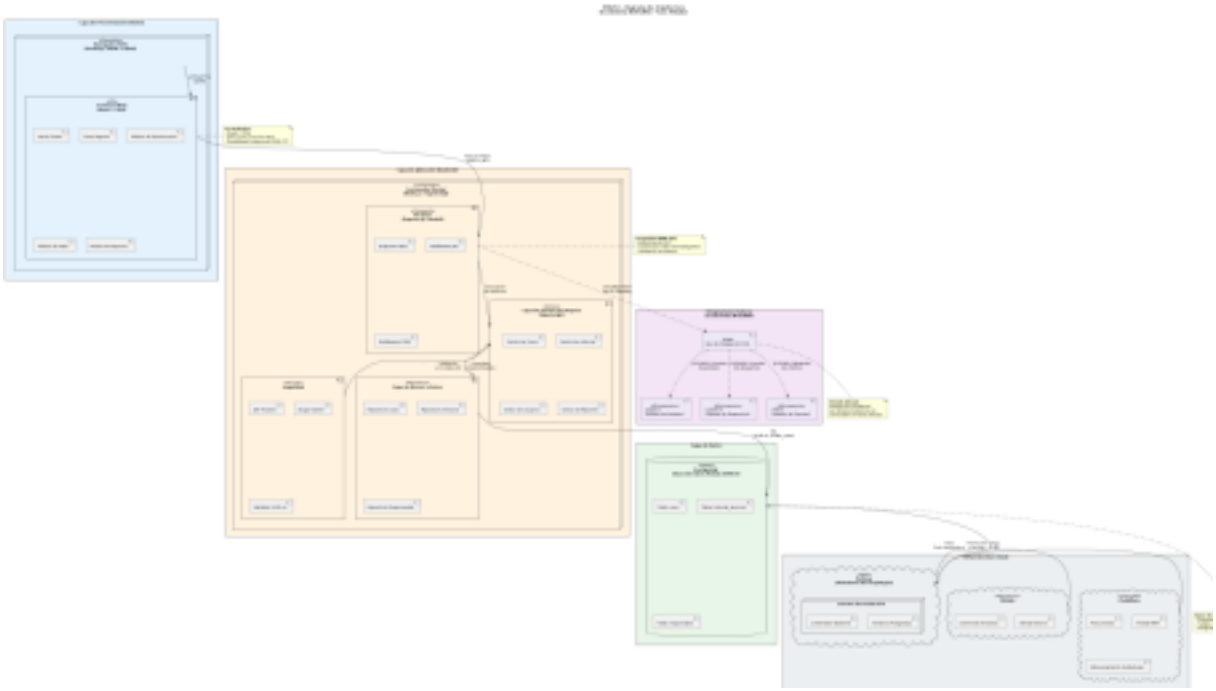


Figura 5. Diagrama de Arquitectura de SERVI-X dentro del ecosistema INTEGRAX.

7. Matriz de Propiedad Intelectual

La siguiente matriz identifica cada componente de software de terceros utilizado en el módulo SERVI-X, su procedencia, su tipo de licencia y el uso específico que se le da dentro del proyecto.

Elemento	Procedencia	Licencia	Uso en el proyecto
React	Meta Platforms, Inc. / comunidad open source	MIT	Librería base del frontend (SPA, componentes de UI).
Vite	Evan You / comunidad open source	MIT	Bundler y servidor de desarrollo del frontend.
react-router-dom	Remix Software / comunidad open source	MIT	Enrutamiento de vistas en el frontend.
Node.js	OpenJS Foundation	MIT	Entorno de ejecución del backend.
Express.js	OpenJS Foundation / comunidad open source	MIT	Framework web para exponer la API REST.
TypeScript	Microsoft Corporation	Apache License 2.0	Lenguaje tipado usado en todo el backend.
PostgreSQL	PostgreSQL Global Development Group	Licencia PostgreSQL (estilo MIT/BSD)	Motor de base de datos relacional del módulo.
Prisma ORM	Prisma Data, Inc. / comunidad open source	Apache License 2.0	Mapeo objeto-relacional y migraciones de base de datos.
bcrypt	Comunidad open source	MIT	Hashing de contraseñas.
jsonwebtoken	Auth0 / comunidad open source	MIT	Emisión y verificación de JSON Web Tokens.
Jest	OpenJS Foundation / Meta	MIT	Framework de pruebas unitarias e integración del backend.
Supertest	Comunidad open source	MIT	Pruebas de integración sobre los endpoints REST.
Docker	Docker, Inc.	Apache License 2.0 (Docker Engine)	Contenerización del backend y la base de datos para despliegue.

Elemento	Procedencia	Licencia	Uso en el proyecto
Código propio del equipo SERVI-X	Equipo desarrollador (Jose Alvarez Marin)	Derechos reservados — trabajo académico de la Corporación Universitaria de Sabaneta	Lógica de negocio, modelos de dominio, controladores, vistas y configuración específica del módulo SERVI-X.

8. Contrato de Licencia de Software

CONTRATO DE LICENCIAMIENTO Y PRESTACIÓN DE DESARROLLO DE SOFTWARE

Identificación del software: el presente contrato tiene por objeto el software denominado **SERVI-X** (módulo de gestión de PQRS del ecosistema INTEGRAX), en su versión documentada en este mismo documento técnico (modelo de datos, diccionario de datos, modelo de clases y arquitectura descritos en las secciones 2 a 6).

Entre los suscritos, a saber: de una parte, **INTEGRAX** (en adelante, "EL CLIENTE"), plataforma empresarial de integración constituida por los seis microproyectos académicos de la Electiva Profesional II — Integración de Aplicaciones y Sistemas de la Corporación Universitaria de Sabaneta; y de otra parte, el equipo desarrollador del módulo **SERVI-X**, representado por **Jose Alvarez Marin** (en adelante, "EL DESARROLLADOR"), se celebra el presente contrato académico de licenciamiento de software, regido por las siguientes cláusulas:

CLÁUSULA PRIMERA — Identificación de las partes. EL DESARROLLADOR es el autor y titular original del software denominado SERVI-X, módulo de gestión de PQRS. EL CLIENTE es la plataforma integradora INTEGRAX, que recibe el derecho de uso del módulo para su incorporación dentro del ecosistema empresarial simulado.

CLÁUSULA SEGUNDA — Objeto del contrato. EL DESARROLLADOR concede a EL CLIENTE una licencia de uso sobre el software SERVI-X, en su versión entregada al cierre de cada hito académico (semanas 1 a 5), para su operación, integración y evaluación dentro de la plataforma INTEGRAX.

CLÁUSULA TERCERA — Titularidad. EL DESARROLLADOR conserva en todo momento los derechos morales y la titularidad intelectual sobre el código fuente original de SERVI-X, de conformidad con la Ley 23 de 1982 sobre derechos de autor. La presente licencia no implica cesión ni transferencia de la propiedad intelectual, sino únicamente la concesión de derechos de uso definidos en este contrato.

CLÁUSULA CUARTA — Derechos de uso. EL CLIENTE podrá instalar, ejecutar, consultar e integrar el software SERVI-X con los demás módulos del ecosistema INTEGRAX (CRM-X, SALES-X, FINAN-X, LOGISTI-X, INVENTA-X), exclusivamente dentro del marco académico de la asignatura Integración de Aplicaciones y Sistemas.

CLÁUSULA QUINTA — Duración. El presente contrato tendrá vigencia desde la fecha de su firma hasta la finalización del periodo académico correspondiente (18 de septiembre de 2026), pudiendo prorrogarse por mutuo acuerdo para fases posteriores de integración real.

CLÁUSULA SEXTA — Territorio. La licencia se concede para uso dentro del territorio académico de la Corporación Universitaria de Sabaneta y, de manera extendida, para cualquier entorno de demostración o evaluación docente asociado al proyecto INTEGRAX.

CLÁUSULA SÉPTIMA — Exclusividad. La presente licencia es de carácter no exclusivo. EL DESARROLLADOR conserva el derecho de reutilizar, adaptar o presentar el código fuente de SERVI-X en otros contextos académicos o profesionales, siempre que no comprometa la confidencialidad de datos propios de EL CLIENTE.

CLÁUSULA OCTAVA — Restricciones. EL CLIENTE no podrá sublicenciar, vender, ni distribuir comercialmente el software SERVI-X a terceros ajenos al ecosistema INTEGRAX sin autorización expresa y escrita de EL DESARROLLADOR.

CLÁUSULA NOVENA — Uso comercial. Cualquier explotación comercial del software, fuera del contexto académico aquí descrito, requerirá la negociación de un nuevo acuerdo de licenciamiento entre las partes, incluyendo la correspondiente contraprestación económica.

CLÁUSULA DÉCIMA — Propiedad intelectual. La propiedad intelectual sobre SERVI-X se distribuye de la siguiente manera: (i) el código propio desarrollado por EL DESARROLLADOR permanece bajo sus derechos reservados conforme a la cláusula tercera; (ii) los componentes de terceros incorporados mantienen la titularidad de sus respectivos autores y se rigen por sus licencias originales. EL CLIENTE no adquiere derecho de propiedad alguno sobre ninguno de los dos, únicamente el derecho de uso aquí concedido.

CLÁUSULA DÉCIMA PRIMERA — Componentes de terceros. EL CLIENTE reconoce que SERVI-X incorpora componentes de software de terceros bajo licencias de código abierto (MIT, Apache License 2.0 y Licencia PostgreSQL, entre otras), según se detalla en la Matriz de Propiedad Intelectual (sección 7 de este documento), y se compromete a respetar los términos de dichas licencias.

CLÁUSULA DÉCIMA SEGUNDA — Distribución y modificación. EL CLIENTE no podrá modificar, descompilar, ni generar obras derivadas del código fuente de SERVI-X sin autorización previa y escrita de EL DESARROLLADOR. Cualquier distribución del software a terceros dentro del ecosistema INTEGRAX deberá conservar los avisos de autoría y de licenciamiento originales.

CLÁUSULA DÉCIMA TERCERA — Soporte y mantenimiento. EL DESARROLLADOR prestará soporte y mantenimiento correctivo sobre el software durante la vigencia del periodo académico (cláusula quinta), limitado a la corrección de errores detectados en las pruebas de integración (sección 10 de este documento). Cualquier

mantenimiento evolutivo o soporte posterior a la finalización del periodo académico requerirá un acuerdo independiente entre las partes.

CLÁUSULA DÉCIMA CUARTA — Terminación. El presente contrato podrá darse por terminado de forma anticipada por incumplimiento de cualquiera de las partes, o de forma ordinaria al concluir el periodo académico pactado, sin perjuicio de la conservación de los derechos morales de EL DESARROLLADOR.

En constancia de lo anterior, se firma el presente contrato académico el 12 de noviembre de 2026.

_____

EL DESARROLLADOR — Jose Alvarez Marin

_____

EL CLIENTE — INTEGRAX (representado académicamente por el docente evaluador)

9. Matriz de Trazabilidad

La siguiente matriz demuestra la cadena lógica de trazabilidad entre cada requisito funcional, su caso de uso de origen, la clase y entidad del modelo de datos que lo materializan, el endpoint REST que lo expone y la prueba de integración diseñada para verificarlo. La numeración RF coincide exactamente con la de la sección 1.5.

RF	Caso de Uso (UC)	Clase	Entidad (MER)	Funcionalidad / API (Endpoint REST)	Prueba
RF-01	UC-01 Crear usuario	Cliente	cliente	POST /api/v1/usuarios	CP-01
RF-02	UC-02 Ingresar	Cliente / Responsable	cliente / responsable	POST /api/v1/auth/login	CP-02, CP-03
RF-03	UC-03 Consultar solicitudes	Caso	caso	GET /api/v1/casos	CP-10
RF-04	UC-04 Consultar historial	HistorialAtencion	historial_atencion	GET /api/v1/casos/{id}/historial	CP-10
RF-05	UC-05 Registrar PQRS	Caso	caso	POST /api/v1/casos	CP-04
RF-06	UC-06 Asignar responsables	Caso	caso	PATCH /api/v1/casos/{id}/asignacion	CP-05
RF-07	UC-07 Actualizar estado	Caso	caso	PATCH /api/v1/casos/{id}/estado	CP-07
RF-08	UC-08 Generar reportes	Caso	caso	GET /api/v1/reportes/resumen	CP-09
RF-09	UC-09 Cerrar casos	Caso	caso	PATCH /api/v1/casos/{id}/cierre	CP-06, CP-08
RF-10	UC-10 Registrar respuestas	HistorialAtencion	historial_atencion	POST /api/v1/casos/{id}/respuesta	CP-07

10. Pruebas de Integración

Las siguientes diez pruebas son de **caja negra**: se ejecutaron interactuando únicamente con la interfaz de usuario de la aplicación real (frontend React en *http://localhost:5173*, backend Express en *http://localhost:3000*, PostgreSQL 16), sin inspeccionar ni depender del código interno, demostrando que los tres componentes (frontend, API y base de datos) se integran y funcionan correctamente de extremo a extremo:

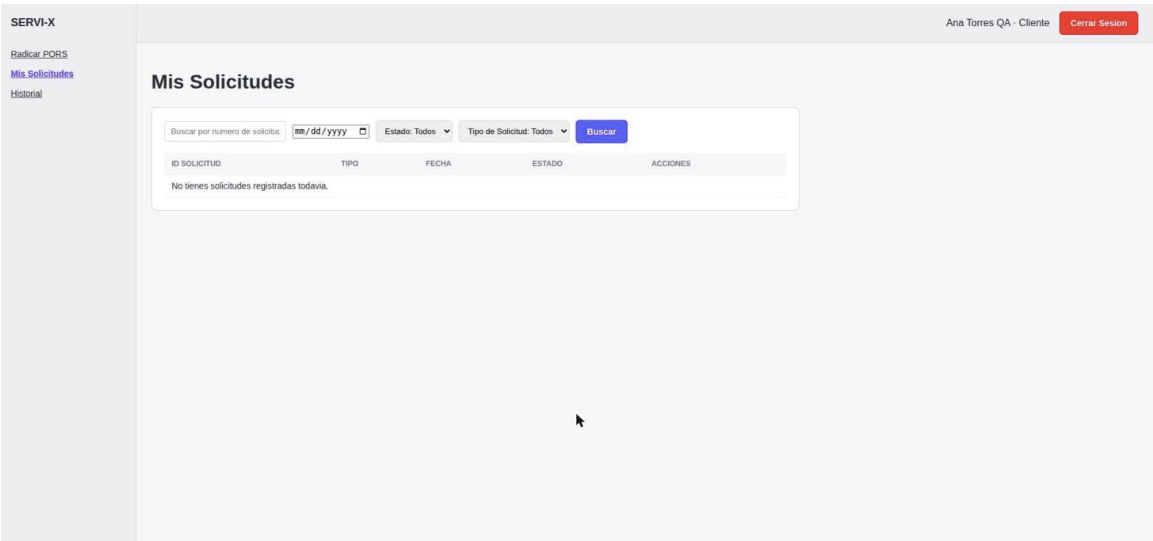
Usuario -> Frontend -> API -> Backend -> Base de datos -> Respuesta -> Frontend -> Usuario

Cada prueba se diseñó con el formato estándar de caso de prueba (objetivo, precondición, datos de entrada, pasos, resultado esperado, resultado obtenido, estado PASS/FAIL, evidencia fotográfica, error encontrado y corrección realizada) y fue ejecutada el 12 de noviembre de 2026.

CP-01 - Crear usuario (Cliente)

Objetivo / RF - UC	RF-01 / UC-01
Precondición	Ninguna. El correo no debe existir previamente en la base de datos.
Datos de entrada	Nombre: Ana Torres QA; Documento: Cedula de Ciudadania 1122334455; Email: ana.torres.qa@example.com; Contraseña: Clave123*
Pasos ejecutados	1) Navegar a /#/registro. 2) Completar el formulario de registro. 3) Clic en “Acceder”.
Resultado esperado	El backend crea el Cliente en PostgreSQL con contraseña hasheada (bcrypt), genera un UUID v4 y un JWT, y el frontend redirige automáticamente a “Mis Solicitudes” ya autenticado.
Resultado obtenido	Igual al esperado: el usuario “Ana Torres QA · Cliente” quedó autenticado en la pantalla “Mis Solicitudes” (ver evidencia).
Estado	PASS
Error encontrado	Ninguno.
Corrección realizada	No aplica.

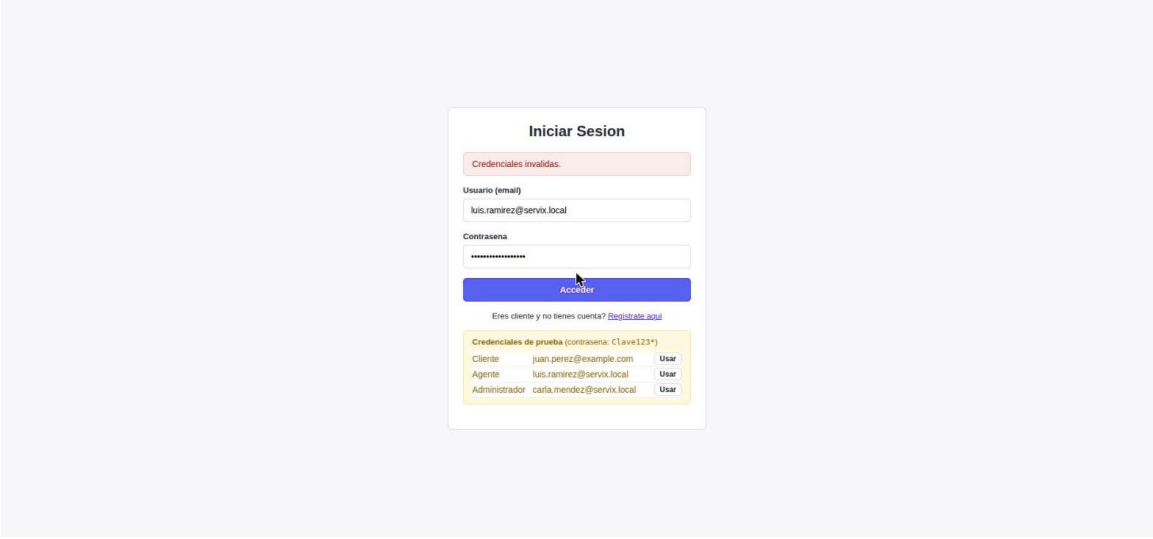
Evidencia (captura de pantalla real):



CP-02 - Rechazo de credenciales inválidas (negativo)

Objetivo / RF - UC	RF-02 / UC-02
Precondición	El responsable luis.ramirez@servix.local existe en el sistema (seed).
Datos de entrada	Email: luis.ramirez@servix.local; Contraseña: ClaveIncorrecta999 (incorrecta).
Pasos ejecutados	1) Navegar a /#/login. 2) Ingresar el email correcto con contraseña incorrecta. 3) Clic en “Acceder”.
Resultado esperado	La API retorna HTTP 401 con el mensaje “Credenciales invalidas.”, sin revelar si el email existe, y el frontend no inicia sesión.
Resultado obtenido	Igual al esperado: se mostró el banner de error “Credenciales invalidas.” y la sesión no se inició (ver evidencia).
Estado	PASS
Error encontrado	Ninguno.
Corrección realizada	No aplica.

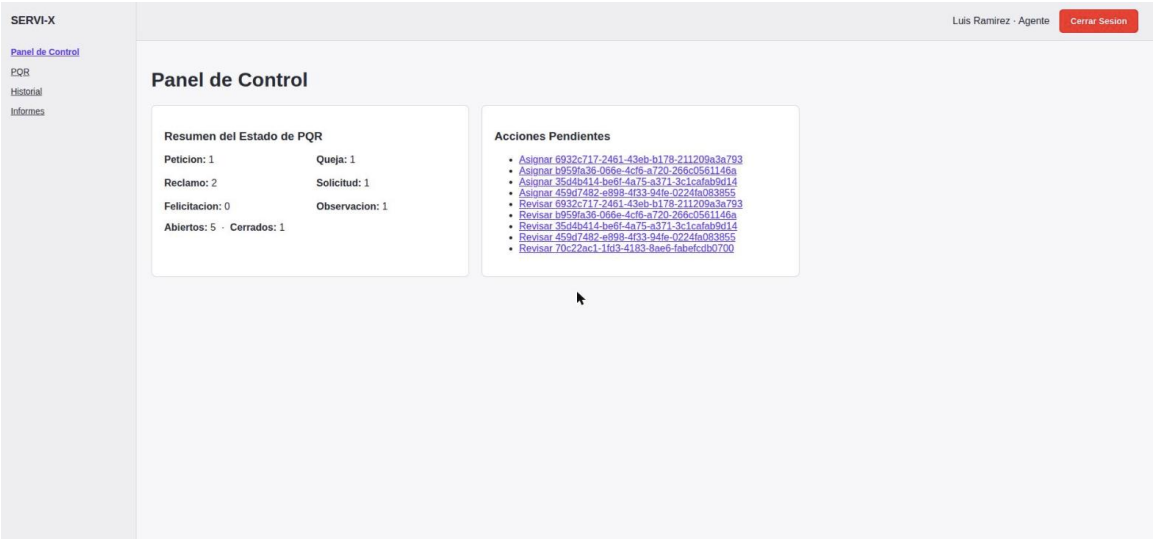
Evidencia (captura de pantalla real):



CP-03 - Ingreso válido de Agente y Panel de Control

Objetivo / RF - UC	RF-02 / UC-02
Precondición	El responsable luis.ramirez@servix.local existe y está activo.
Datos de entrada	Email: luis.ramirez@servix.local; Contraseña: Clave123*.
Pasos ejecutados	1) Navegar a /#/login. 2) Usar el botón “Usar” de la fila Agente. 3) Clic en “Acceder”.
Resultado esperado	Autenticación exitosa; redirección al Panel de Control con el resumen de casos agrupado por los 6 tipos de solicitud definidos en UC-05 y el conteo de abiertos/cerrados.
Resultado obtenido	Igual al esperado. El panel mostró “Luis Ramirez · Agente” y el resumen con las 6 categorías (Peticion, Queja, Reclamo, Solicitud, Felicitacion, Observacion).
Estado	PASS
Error encontrado	Ninguno.
Corrección realizada	No aplica.

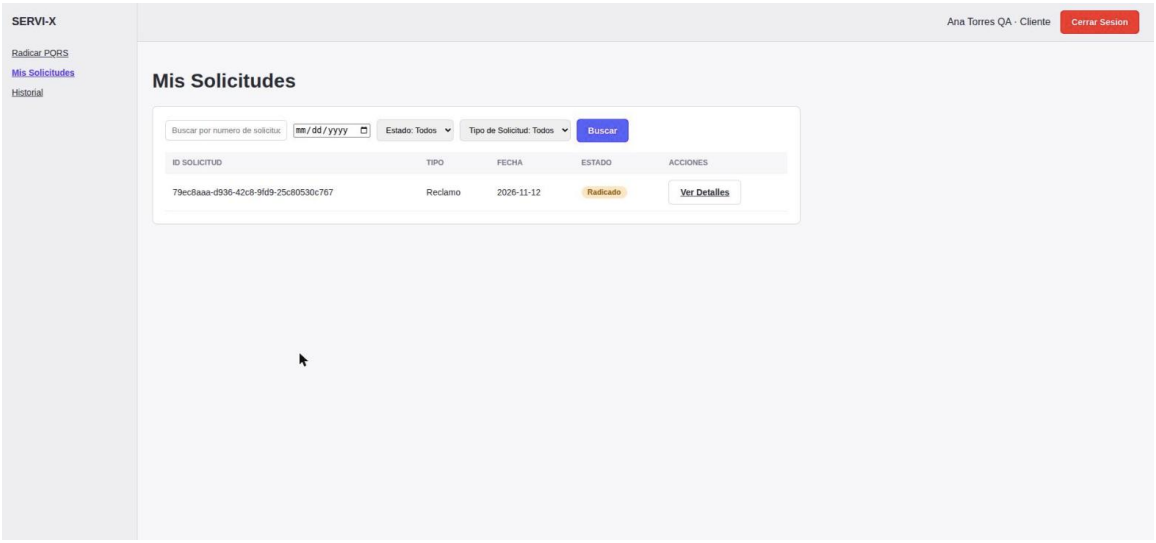
Evidencia (captura de pantalla real):



CP-04 - Registrar PQRS

Objetivo / RF - UC	RF-05 / UC-05
Precondición	Cliente autenticado (Ana Torres QA, de CP-01).
Datos de entrada	Tipo: Reclamo; Descripcion: “El producto llego con el empaque roto, solicito reemplazo.”; Numero de Pedido asociado: SALES-X-QA-01 (texto libre, no UUID).
Pasos ejecutados	1) Menú “Radicar PQRS”. 2) Completar tipo, descripción y pedido asociado. 3) Clic en “Enviar PQRS”.
Resultado esperado	Se crea el caso en PostgreSQL con estado “Radicado” y un registro inicial automático en el historial (RN-04); el campo pedido asociado acepta texto libre sin exigir formato UUID.
Resultado obtenido	Igual al esperado: el caso quedó visible en “Mis Solicitudes” con estado “Radicado”.
Estado	PASS
Error encontrado	Ninguno.
Corrección realizada	No aplica.

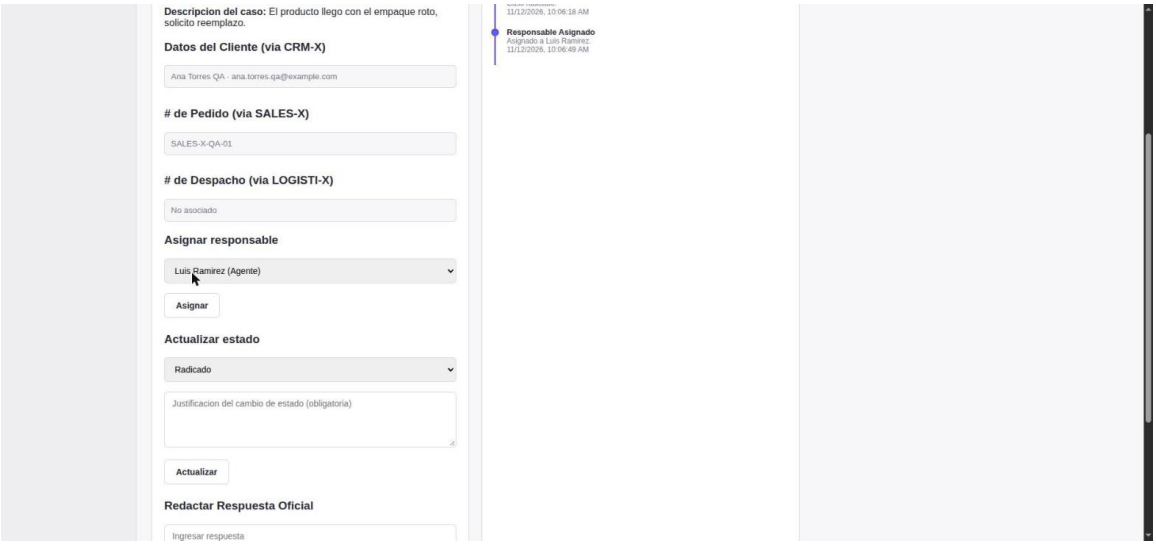
Evidencia (captura de pantalla real):



CP-05 - Asignar responsable

Objetivo / RF - UC	RF-06 / UC-06
Precondición	Caso de CP-04 en estado Radicado; Agente autenticado.
Datos de entrada	Responsable seleccionado: Luis Ramirez (Agente).
Pasos ejecutados	1) Como Agente, abrir “PQR”. 2) Abrir el detalle del caso de Ana Torres QA. 3) Seleccionar responsable y clic en “Asignar”.
Resultado esperado	El caso queda asignado a Luis Ramirez; se registra el evento “Responsable Asignado” en la línea de tiempo del historial.
Resultado obtenido	Igual al esperado: el timeline del caso muestra “Responsable Asignado – Asignado a Luis Ramirez.” con fecha y hora exactas.
Estado	PASS
Error encontrado	Ninguno.
Corrección realizada	No aplica.

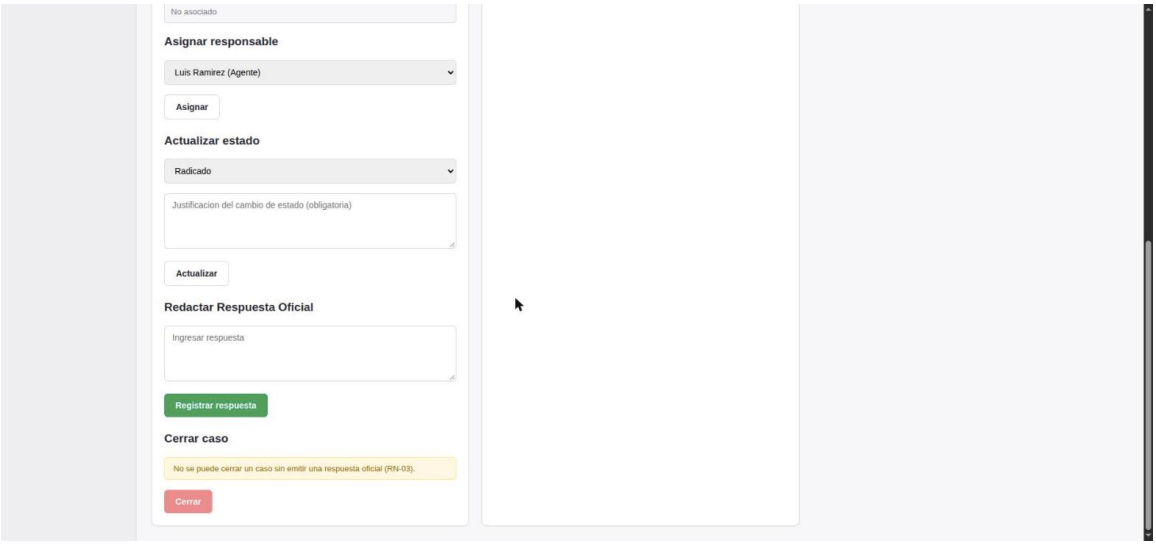
Evidencia (captura de pantalla real):



CP-06 - Bloqueo de cierre sin respuesta oficial (negativo, RN-03)

Objetivo / RF - UC	RF-09 / UC-09 / RN-03
Precondición	Caso asignado, sin ningún evento “RespuestaOficial” en su historial.
Datos de entrada	Ninguno (solo se intenta la acción de cierre).
Pasos ejecutados	1) En el detalle del caso, intentar hacer clic en “Cerrar”.
Resultado esperado	El botón “Cerrar” aparece deshabilitado y se muestra el mensaje “No se puede cerrar un caso sin emitir una respuesta oficial (RN-03).”; la API rechazaría la operación con HTTP 400 si se forzara.
Resultado obtenido	Igual al esperado: el mensaje de bloqueo se mostró y el botón “Cerrar” quedó deshabilitado (rojo claro).
Estado	PASS
Error encontrado	Ninguno.
Corrección realizada	No aplica.

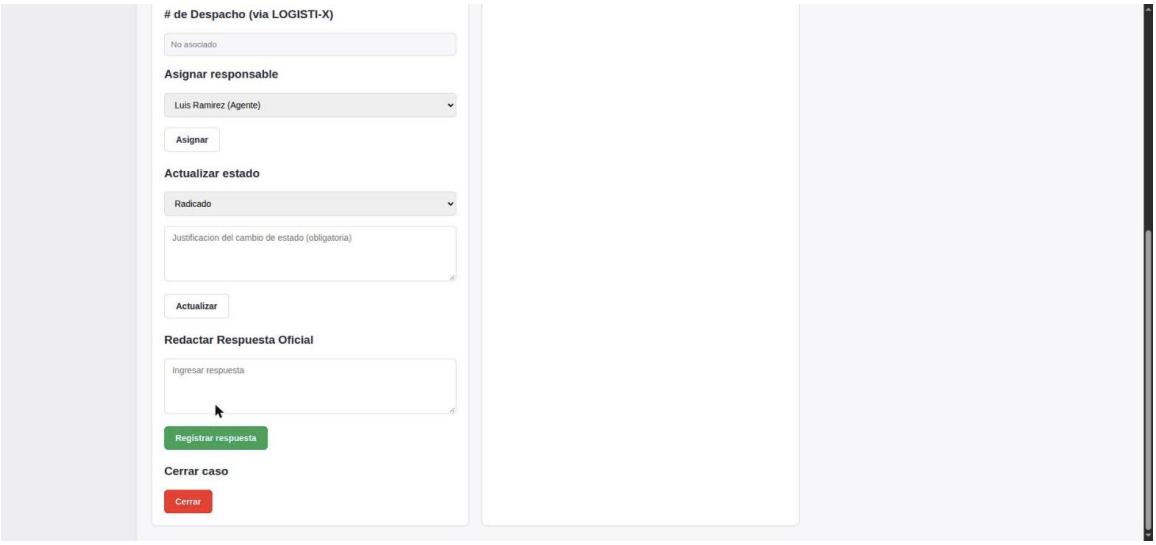
Evidencia (captura de pantalla real):



CP-07 - Registrar respuesta oficial

Objetivo / RF - UC	RF-10 / UC-10
Precondición	Caso asignado (continuación de CP-06).
Datos de entrada	Respuesta: “Hemos revisado su reclamo y procederemos a enviar un reemplazo del producto sin costo adicional.”
Pasos ejecutados	1) Escribir el texto en “Redactar Respuesta Oficial”. 2) Clic en “Registrar respuesta”.
Resultado esperado	Se inserta el evento “RespuestaOficial” en el historial, sin pedir el correo del cliente (se resuelve automáticamente desde el caso); el botón “Cerrar” se habilita.
Resultado obtenido	Igual al esperado: tras guardar, el botón “Cerrar” cambió a color rojo sólido (habilitado).
Estado	PASS
Error encontrado	Ninguno.
Corrección realizada	No aplica.

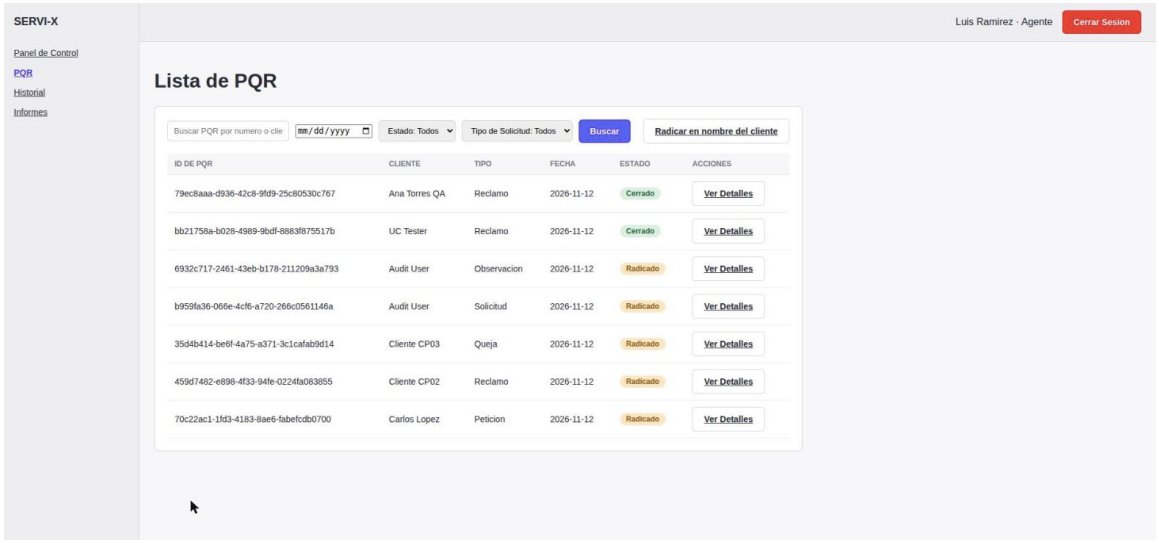
Evidencia (captura de pantalla real):



CP-08 - Cierre exitoso de caso

Objetivo / RF - UC	RF-09 / UC-09
Precondición	Caso con respuesta oficial registrada (CP-07).
Datos de entrada	Ninguno (solo se ejecuta la acción de cierre).
Pasos ejecutados	1) Clic en “Cerrar”.
Resultado esperado	El caso cambia a estado “Cerrado”, se estampa fechaCierre, y queda bloqueado contra modificaciones futuras.
Resultado obtenido	Igual al esperado: en la lista de PQR, el caso de Ana Torres QA aparece con estado “Cerrado” (verde).
Estado	PASS
Error encontrado	Ninguno.
Corrección realizada	No aplica.

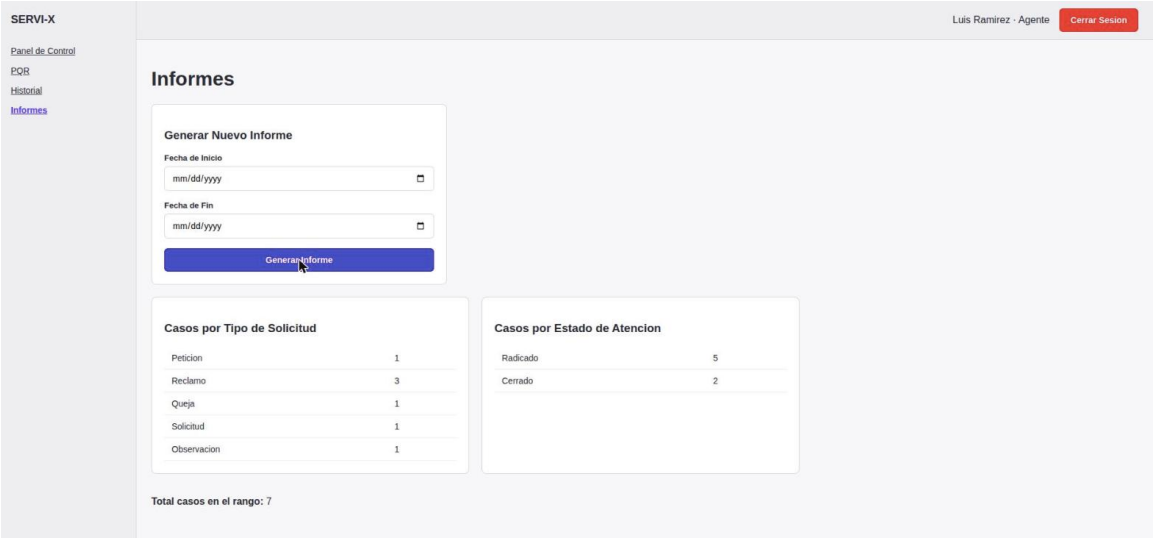
Evidencia (captura de pantalla real):



CP-09 - Generar reportes de gestión

Objetivo / RF - UC	RF-08 / UC-08
Precondición	Agente autenticado; existen casos de distintos tipos y estados en el sistema.
Datos de entrada	Sin filtro de fecha (rango abierto).
Pasos ejecutados	1) Menú “Informes”. 2) Clic en “Generar Informe”.
Resultado esperado	El sistema agrupa los casos por tipo de solicitud y por estado de atención mediante una consulta GROUP BY en PostgreSQL, mostrando el total.
Resultado obtenido	Igual al esperado: se mostraron las tablas “Casos por Tipo de Solicitud” y “Casos por Estado de Atencion” con el total de 7 casos.
Estado	PASS
Error encontrado	Ninguno.
Corrección realizada	No aplica.

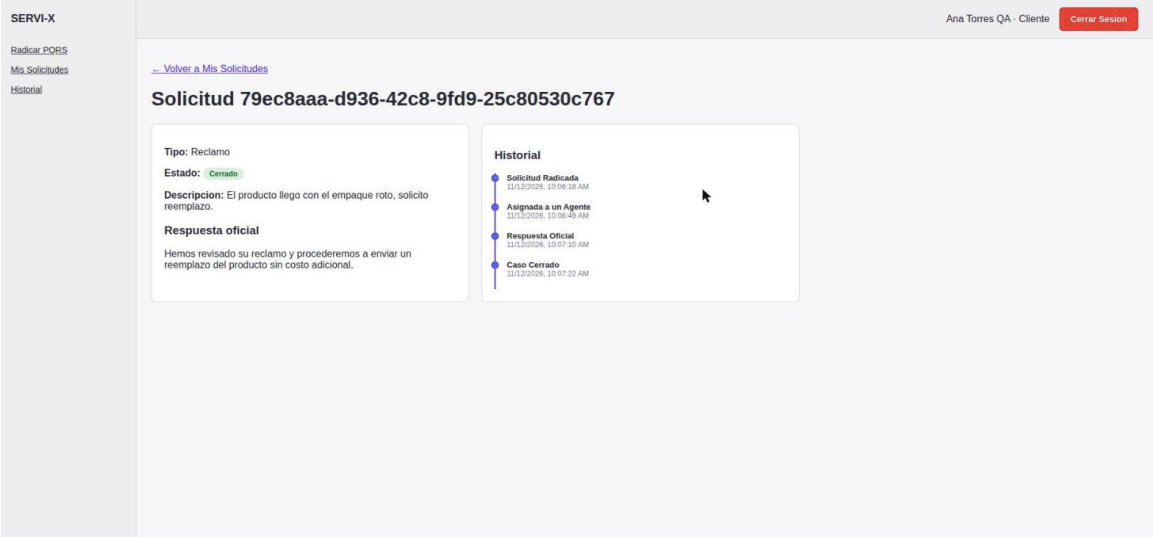
Evidencia (captura de pantalla real):



CP-10 - El Cliente consulta la respuesta oficial de su caso (cierre del ciclo)

Objetivo / RF - UC	RF-03, RF-04 / UC-03, UC-04
Precondición	Caso cerrado con respuesta oficial (CP-08).
Datos de entrada	Ninguno.
Pasos ejecutados	1) Iniciar sesión como Ana Torres QA (Cliente). 2) Abrir “Mis Solicitudes”. 3) Clic en “Ver Detalles” del caso.
Resultado esperado	El Cliente visualiza el estado “Cerrado”, el texto completo de la respuesta oficial, y la línea de tiempo completa (Solicitud Radicada, Asignada a un Agente, Respuesta Oficial, Caso Cerrado), confirmando la integración de extremo a extremo entre frontend, API y base de datos.
Resultado obtenido	Igual al esperado.
Estado	PASS
Error encontrado	Ninguno.
Corrección realizada	No aplica.

Evidencia (captura de pantalla real):



11. Anexo: Script SQL del Esquema de Base de Datos

El siguiente script corresponde al esquema exacto implementado en PostgreSQL 16, obtenido mediante *pg_dump --schema-only* sobre la base de datos real del módulo (sin datos, solo definición de estructura). Se omite la tabla interna _prisma_migrations, propia del ORM, por no formar parte del modelo de dominio.

```
-- Tipo enumerado para el rol operativo de los responsables (Agente, Administrador)
CREATE TYPE public."Rol" AS ENUM (
'Agente',
'Administrador'
);

-- Funcion que implementa la regla append-only de historial_atencion (RNF-05)
CREATE FUNCTION public.historial_atencion_bloquear_mutacion() RETURNS trigger
LANGUAGE plpgsql
AS $$
BEGIN
RAISE EXCEPTION 'historial_atencion es append-only: no se permiten UPDATE ni DELETE';
END;
$$;

-- Entidad Cliente (usuario final que radica PQRS, UC-01)
CREATE TABLE public.cliente (
cliente_id uuid NOT NULL,
nombre_completo character varying(150) NOT NULL,
tipo_documento character varying(50) NOT NULL,
numero_documento character varying(50) NOT NULL,
email character varying(100) NOT NULL,
password_hash character varying(255) NOT NULL,
fecha_creacion timestamp with time zone DEFAULT CURRENT_TIMESTAMP NOT NULL
);

-- Entidad Responsable (agentes y administradores internos)
CREATE TABLE public.responsable (
responsable_id uuid NOT NULL,
nombre_completo character varying(150) NOT NULL,
correo_corporativo character varying(100) NOT NULL,
password_hash character varying(255) NOT NULL,
rol_operativo public."Rol" NOT NULL,
estado_activo boolean DEFAULT true NOT NULL
);

-- Entidad Caso (nucleo transaccional de la PQRS)
CREATE TABLE public.caso (
caso_id uuid NOT NULL,
cliente_id uuid NOT NULL,
pedido_id character varying(100),
despacho_id character varying(100),
tipo_solicitud character varying(50) NOT NULL,
descripcion text NOT NULL,
estado_atencion character varying(50) DEFAULT 'Radicado'::character varying NOT NULL,
responsable_id uuid,
fecha_creacion timestamp with time zone DEFAULT CURRENT_TIMESTAMP NOT NULL,
fecha_cierre timestamp with time zone
);

-- Entidad Historial_Atencion (bitacora inmutable de auditoria, RN-04)
CREATE TABLE public.historial_atencion (
historial_id uuid NOT NULL,
```

```
caso_id uuid NOT NULL,
estado_anterior character varying(50),
estado_nuevo character varying(50) NOT NULL,
accion_realizada text NOT NULL,
usuario_accion_id uuid NOT NULL,
accion character varying(50) NOT NULL,
fecha_registro timestamp with time zone DEFAULT CURRENT_TIMESTAMP NOT NULL
);

-- Llaves primarias
ALTER TABLE ONLY public.cliente
ADD CONSTRAINT cliente_pkey PRIMARY KEY (cliente_id);
ALTER TABLE ONLY public.responsable
ADD CONSTRAINT responsable_pkey PRIMARY KEY (responsable_id);
ALTER TABLE ONLY public.caso
ADD CONSTRAINT caso_pkey PRIMARY KEY (caso_id);
ALTER TABLE ONLY public.historial_atencion
ADD CONSTRAINT historial_atencion_pkey PRIMARY KEY (historial_id);

-- Indices unicos (RN-01 / unicidad de credenciales)
CREATE UNIQUE INDEX cliente_email_key ON public.cliente USING btree (email);
CREATE UNIQUE INDEX responsable_correo_corporativo_key ON public.responsable USING btree
(correo_corporativo);

-- Trigger append-only (RNF-05): bloquea UPDATE y DELETE sobre historial_atencion
CREATE TRIGGER trg_historial_atencion_append_only
BEFORE DELETE OR UPDATE ON public.historial_atencion
FOR EACH ROW EXECUTE FUNCTION public.historial_atencion_bloquear_mutacion();

-- Llaves foraneas
ALTER TABLE ONLY public.caso
ADD CONSTRAINT caso_cliente_id_fkey FOREIGN KEY (cliente_id)
REFERENCES public.cliente(cliente_id) ON UPDATE CASCADE ON DELETE RESTRICT;
ALTER TABLE ONLY public.caso
ADD CONSTRAINT caso_responsable_id_fkey FOREIGN KEY (responsable_id)
REFERENCES public.responsable(responsable_id) ON UPDATE CASCADE ON DELETE SET NULL;
ALTER TABLE ONLY public.historial_atencion
ADD CONSTRAINT historial_atencion_caso_id_fkey FOREIGN KEY (caso_id)
REFERENCES public.caso(caso_id) ON UPDATE CASCADE ON DELETE RESTRICT;
```

Documento elaborado a partir de la especificación funcional, la especificación técnica y el modelado de la solución (UML, MER, diccionario de datos, secuencia y arquitectura) para el módulo SERVI-X dentro del proyecto INTEGRAX. Versión 3, actualizada el 12 de noviembre de 2026, con verificación funcional completa (frontend + API + PostgreSQL) y pruebas de integración de caja negra con evidencia real.